

[Название учебного заведения]

Отчёт по практике

Разработка веб приложения Notable

REST API и браузерный интерфейс для заметок

Студент: [Студент]

Группа: [Группа]

Руководитель: [Руководитель]

Сроки практики: [Сроки практики]

[Город] · 2026

1 Выбранный проект

Create a Django API in Under 20 Minutes — учебный проект API заметок по статье Scott Domes. Приложение позволяет сохранять заголовок и текст, просматривать записи, изменять и удалять их.

[Статья с выбранным проектом](#)

[Проект в каталоге Python](#)

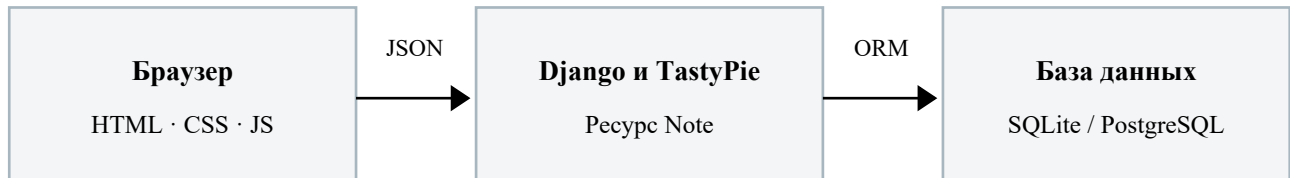
Сохранены Django, TastyPie, модель Note и маршруты `/api/note/`. Добавлены браузерный интерфейс, проверка данных, страницы списка, тесты, конфигурация Render и материалы для демонстрации.

2 Технический паспорт проекта

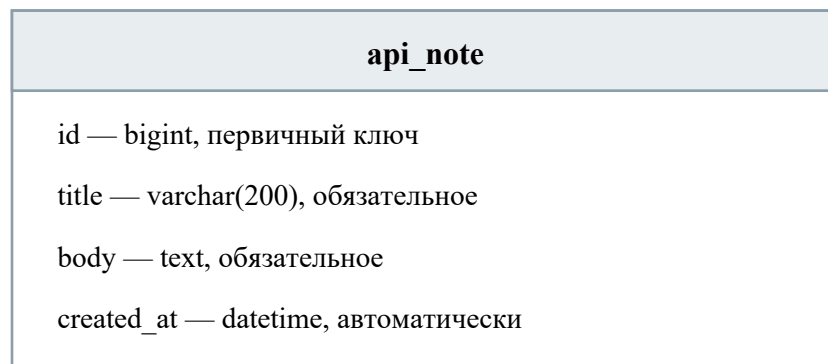
Параметр	Значение
Название	Notable
GitHub	Публичный репозиторий [после публикации]
Деплой	Публичный сайт [после публикации]
Frontend	HTML, CSS, JavaScript, Django Templates
Backend	Python; Django 5.2.17; TastyPie 0.15.1
База данных	SQLite при локальном запуске; PostgreSQL в конфигурации Render
Развёртывание	Render Blueprint, Gunicorn, WhiteNoise
Доступы	Login/password не требуются. Общая доска без регистрации. Админка отключена.
Демонстрация	docs/demo.gif — 24,5 секунды [после публикации]
Локальная проверка	Python 3.14.7; 8 тестов пройдены; Django check без ошибок

3 Архитектура

Клиентская страница обращается к API по HTTP. Django маршрутизирует запрос, TastyPie преобразует его в операцию над Note, а ORM выполняет запрос к базе данных. Ответы API сериализуются в JSON.



ERD предметной области



Предметная область содержит одну таблицу без внешних ключей. У заметки нет владельца: данные общие. Служебные таблицы Django для миграций, сессий и встроенной системы пользователей не являются дополнительными сущностями блокнота.

Обработка данных

NoteForm проверяет обязательность title и body и максимальную длину заголовка. created_at устанавливается при сохранении и скрыт в API. Вывод заметок через textContent не превращает пользовательский текст в HTML. Разрешены только нужные методы: список GET/POST и запись GET/PUT/DELETE.

3 Архитектура и сценарии использования

Use Case	Действие и ожидаемый результат
Создать	Посетитель вводит заголовок и текст. POST создаёт запись; карточка появляется в списке.
Прочитать	Посетитель открывает список или URL записи. GET возвращает сохранённые данные.
Изменить	Кнопка «Изменить» заполняет форму. PUT сохраняет новый текст; список обновляется.
Удалить	Посетитель подтверждает удаление. DELETE удаляет запись; следующий GET возвращает 404.

Контракт API

Метод и маршрут	Входные данные	Успех
GET /api/note/	limit, offset — параметры страницы	200, meta и objects
POST /api/note/	JSON с title и body	201, Location
GET /api/note/{id}/	ID в адресе	200, title, body, resource_uri
PUT /api/note/{id}/	JSON с title и body	204, пустое тело
DELETE /api/note/{id}/	ID в адресе	204, пустое тело

Пример создания

```
POST /api/note/
Content-Type: application/json
{"title": "План", "body": "Подготовить демонстрацию"}
```

Пример ответа на чтение

```
{"title": "План", "body": "Подготовить демонстрацию", "resource_uri": "/api/note/1/"}
```

ID 1 приведён как пример. Фактический адрес берётся из Location или resource_uri. При неверных данных возвращается 400, при отсутствии записи — 404, при запрещённом методе — 405. Конечный слеш обязателен. Коллекция Postman находится в корне репозитория.

4 Таблица соответствия реализации

Таблица связывает каждую пользовательскую функцию с исходниками и местом её проверки. Ссылки на исходники и страницы формируются из адресов, указанных в техническом паспорте.

Функция	Исходники GitHub	Экран или маршрут
Просмотр и страницы списка	api/static/api/app.js [после публикации]	/#notes [после публикации] GET /api/note/?limit=6
Создание заметки	api/resources.py [после публикации]	/#create [после публикации] POST /api/note/
Чтение одной заметки	api/resources.py [после публикации]	/api/note/1/ [после публикации] GET /api/note/{id}/
Изменение заметки	api/static/api/app.js [после публикации]	/#create [после публикации] PUT /api/note/{id}/
Удаление с подтверждением	api/static/api/app.js [после публикации]	/#notes [после публикации] DELETE /api/note/{id}/
Проверка обязательных полей	api/resources.py [после публикации]	/#create [после публикации] HTTP 400 при неверных данных
Сохранение в базе данных	api/models.py [после публикации]	/#notes [после публикации] Данные остаются после обновления
Адаптивная страница	api/static/api/styles.css [после публикации]	/ [после публикации] Одна колонка на узком экране

Для API одной записи используйте существующий ID из GET /api/note/. Если начальная заметка удалена, пример /api/note/1/ закономерно вернёт 404. Сценарий изменения начинается кнопкой на карточке; #create указывает на форму редактирования.

Поддерживающие файлы

Миграция таблицы [после публикации]

Автоматические тесты [после публикации]

Конфигурация развёртывания [после публикации]

5 Демонстрация работы

Формат: GIF, 24,5 секунды. Показан основной сценарий: заполнение формы → создание заметки → изменение текста → сохранённый результат. Изображения получены из работающего локального приложения.

Открыть демонстрацию docs/demo.gif [после публикации]

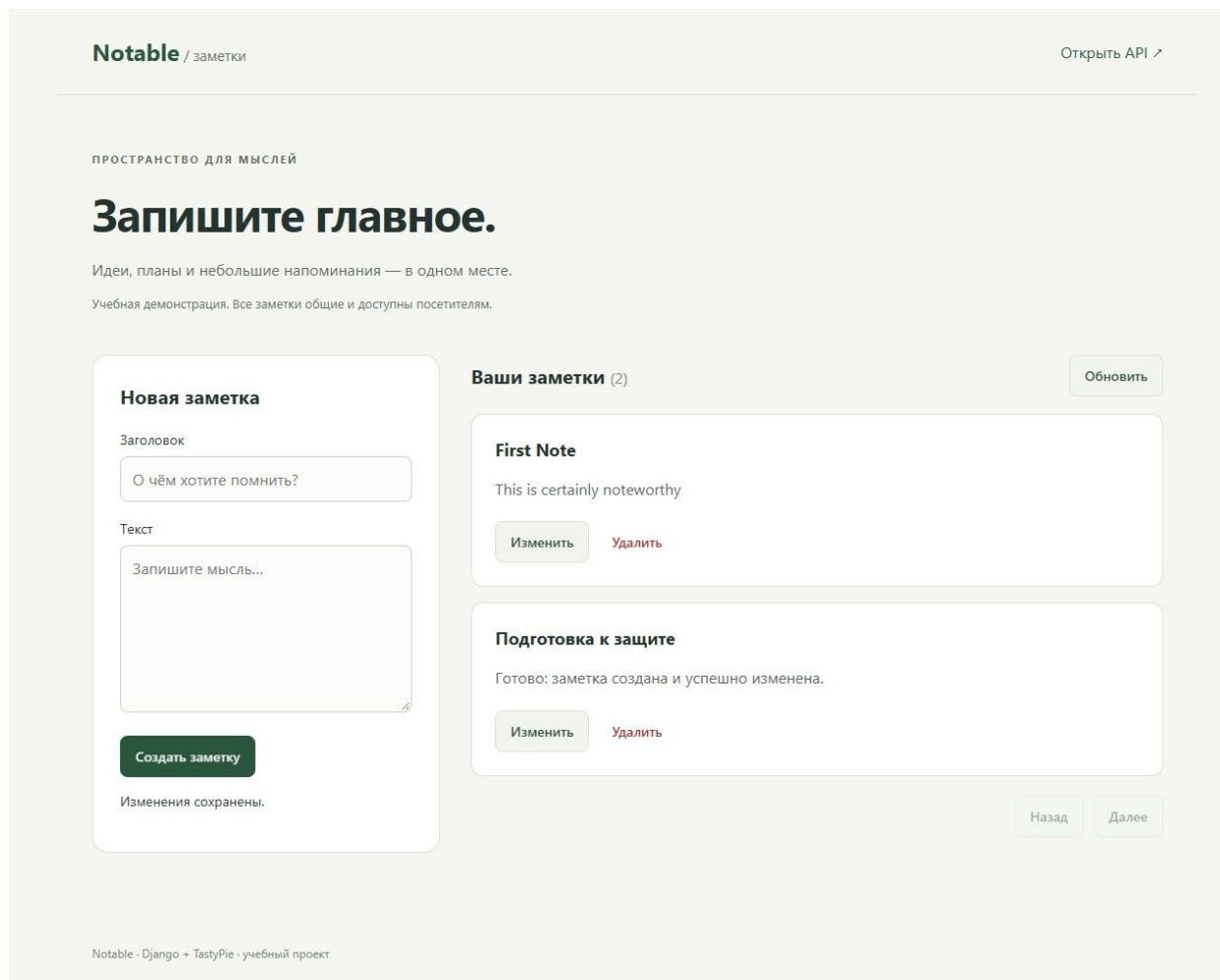


Рисунок 1 — заметка после изменения текста

Для повторения открыть главную страницу, создать заметку «Подготовка к защите», изменить текст на «Готово: заметка создана и успешно изменена.» и сохранить. Карточка должна отобразить новый текст. После публикации сценарий повторяется в режиме инкогнито на публичном адресе.

6 Качество кода

Локальная проверка завершилась успешно: восемь тестов Django пройдены. Проверка конфигурации не выявила ошибок, новых незаписанных миграций нет. Автоматическая проверка в GitHub Actions настроена на push и pull request; её результат появится после загрузки репозитория.

Проверка	Подтверждённый результат
CRUD и хранение данных	Создание 201; чтение 200; изменение и удаление 204; после удаления 404.
Некорректный ввод	Отсутствующее поле, пустой текст, заголовок длиннее 200 символов и неправильный JSON отклоняются.
Безопасность изменения	Ошибка валидации не меняет прежнее содержимое записи. Массовое удаление запрещено.
Список и интерфейс	Пагинация выдаёт следующую страницу; главная страница содержит форму. Админка недоступна по умолчанию.
Браузер	Создание и изменение записи проверены вручную; результат отражён в GIF.

Внешний анализ качества

Сервис: не подключён. Оценка: не подтверждена. Отчёт сервиса [после публикации]

Требование гайда — бейдж Code Climate с оценкой A или B. В официальной документации Code Climate Quality новые пользователи направляются в Qlty. При использовании Qlty замена должна быть согласована с преподавателем. Успешные тесты и файл конфигурации не заменяют результат внешнего анализа.

[Источник о переходе Code Climate Quality на Qlty](#)

Согласование замены сервиса: не отмечено в данных отчёта.

7 Вывод по практике

В результате работы подготовлено приложение Notable для хранения заметок. В основе проекта лежит учебный пример создания Django API. Его функциональность сохранена: запись содержит заголовок, текст и время создания, а клиент может создавать, получать, изменять и удалять записи. Проект дополнен страницей для работы через браузер, автоматическими проверками и материалами для развёртывания. Благодаря этому основной сценарий можно продемонстрировать без установки Postman у проверяющего.

Работа с проектом показывает, как распределяются обязанности между клиентской и серверной частями. Браузер собирает введённые данные и отправляет JSON. Маршрутизация Django передаёт запрос ресурсу TastyPie. Ресурс проверяет данные и выполняет действие над моделью, а ORM обращается к базе. Интерфейс получает результат и обновляет список. При такой организации одна и та же серверная логика используется и страницей заметок, и внешним HTTP-клиентом. Изменение оформления не требует изменения структуры таблицы.

Одной из технических задач стала адаптация исходного учебного материала к установленной версии Django. Вместо старого механизма объявления маршрутов применён актуальный интерфейс `django.urls`. Структура проекта разделена на настройки, модель, ресурс, шаблон и статические файлы. Миграция хранится вместе с исходниками: после установки зависимостей другой разработчик может создать таблицу одной командой. Демонстрационные данные добавляются отдельно, причём повторный запуск команды не создаёт одинаковую начальную запись.

Другой важной задачей была согласованная обработка ошибок. Ограничения HTML помогают посетителю заполнить форму, но сервер выполняет собственную проверку заголовка и текста. Невалидный запрос не должен менять уже сохранённую запись. Для этой ситуации добавлен отдельный тест. Ответы 201 и 204 обрабатываются без попытки разобрать отсутствующее тело JSON. Вывод текста заметок через `textContent` позволяет показывать пользовательский ввод как текст, а не исполнять его в качестве разметки.

Проверка проекта включала полный цикл CRUD, запрос отсутствующей записи, ошибочный JSON, пустые поля, ограничение длины заголовка и разбиение списка на страницы. Восемь автоматических тестов завершились успешно. В браузере дополнительно проверены создание и изменение заметки; эти действия отражены в демонстрационном GIF. Автотесты полезны тем, что проверяют результат запроса и состояние базы, а не только внешний вид страницы. При этом локальная проверка не заменяет контроль доступности будущего публичного адреса.

Подготовка к размещению потребовала учесть отличие локальной среды от хостинга. Для локального запуска используется SQLite, для Render предусмотрено подключение PostgreSQL через переменную окружения. Секрет приложения, разрешённые хосты и режим отладки вынесены в настройки среды. Статические файлы собираются отдельной командой. Эти решения делают процесс запуска воспроизводимым и позволяют хранить исходники отдельно от секретов и рабочих данных. Публикация и проверка на хостинге выполняются после настройки аккаунта владельца.

Дальнейшее развитие проекта может включать персональные аккаунты, разграничение доступа к заметкам, поиск, теги и восстановление удалённых записей. Сейчас приложение представляет собой общую учебную доску: все посетители работают с одними данными. Для использования в качестве личного блокнота сначала потребуется реализовать владельца записи и проверку прав на каждую операцию. Ещё одно направление развития — автоматические проверки интерфейса и анализ качества кода внешним сервисом. Реальный результат такого анализа необходимо подтверждать ссылкой на отчёт.